Raffinamento dei requisiti

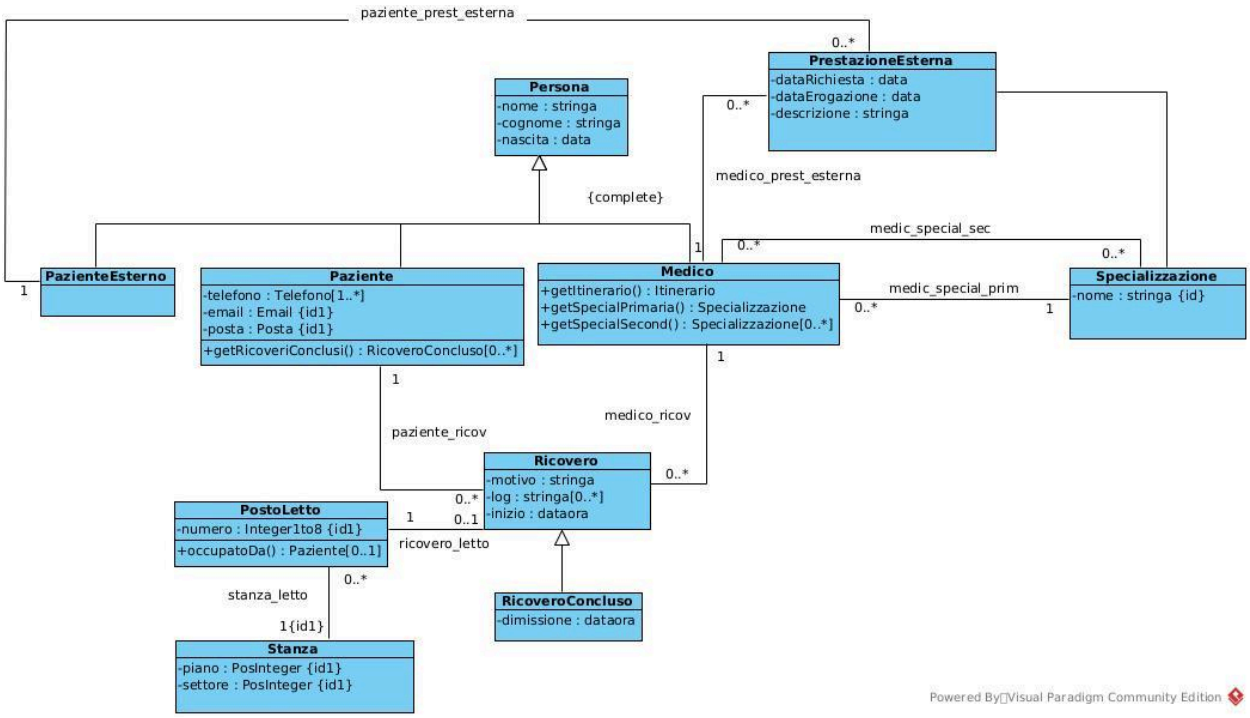
1. Paziente dell'ospedale
    a. Nome e cognome
    b. Data di nascita
    c. Recapiti telefonici
    d. Recapiti email
        i. Sono unici tra tutti i pazienti
    e. Recapiti postali
        i. Sono unici tra tutti i pazienti ?
2. Medico
    a. Nome e cognome
    b. Data di nascita
    c. Specializzazione primaria(vedi req. 8)
    d. Specializzazioni secondarie (vedi req. 8)
    e. I pazienti che hanno in cura (vedi req. 1)
3. Stanze
    a. Piano (intero positivo)
    b. Settore (intero positivo) #è più un semplice numero di stanza
    c. Numero di posti letto (tra 1 e 8)
4. Posto letto
    a. Stanza in cui si trova (vedi req. 3)
    b. Se disponibile
        i. Se in uso, interessa il paziente in ricovero che lo sta usando
5. Ricovero
    a. Paziente (Vedi req.1)
    b. Medico responsabile del paziente (vedi req.2)
    c. Posto letto usato dal paziente (vedi req. 4)
    d. Motivo del ricovero (stringa)
    e. *Note sull'andamento del ricovero (stringhe)
    f. Inizio (data)
    g. Se è gia concluso
        i. In quel caso, interessa la data di dimissione

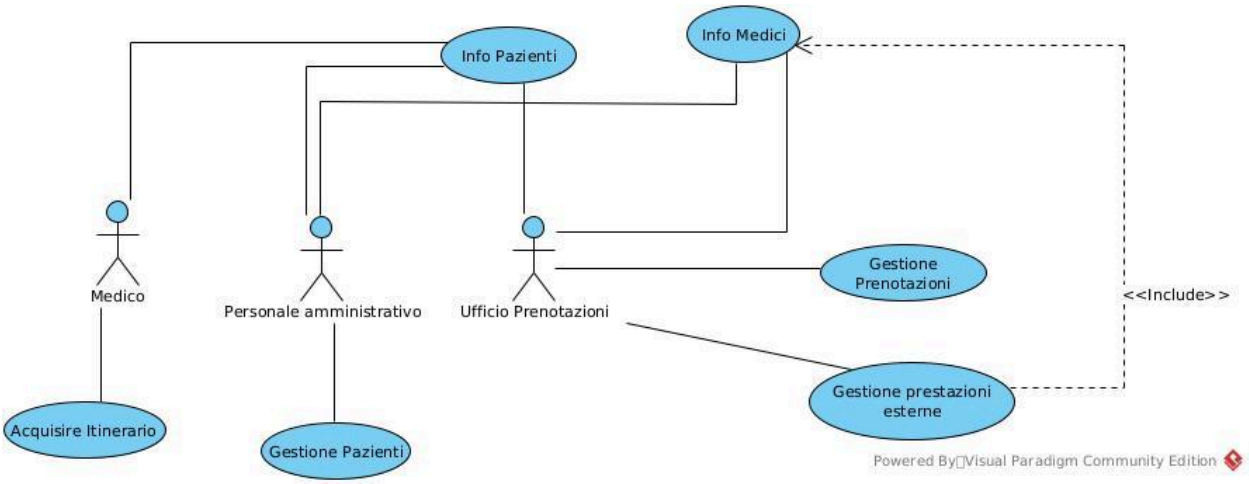
6. Paziente esterno
    a. Nome e cognome
    b. Data di nascita
    c. Recapiti telefonici
    d. Recapiti email
    e. Recapiti postali
        i. Sono unici tra tutti i pazienti
    f. Le prestazioni mediche richieste (vedi req.7)
7. Prestazione medica esterna
    a. Quando è stata richiesta (data)
    b. Quando è stata erogata (data)
    c. Specializzazione richiesta (vedi req. 8)
    d. *Medico responsabile (vedi req. 2)
    e. Descrizione estesa
8. Specializzazione
    a. Nome (stringa)
99. Funzionalità richieste al sistema
    a. Il sistema è accessibile da medici, ufficio prenotazioni, e personale amministrativo
    b. Ufficio prenotazioni deve poter prenotare un posto letto per il ricovero di un paziente in una certa data
        i. Il sistema deve rifiutare la prenotazione se non ci sono posti letto disponibili per tale date
    c. Il sistema deve poter ottenere lo storico di tutti i pazienti che sono stati ricoverati e poi dimessi nel tempo con le informazioni relative ai posti letto occupati durante i diversi ricoveri (accessibile da tutti e 3 gli attori?)
    d. Il personale di accettazione (ufficio prenotazioni??) deve poter registrare un nuovo paziente, il suo ricovero, e la dimissione del ricovero
    e. I medici devono poter richiedere al sistema la creazione di un itinerario, ovvero una LISTA ORDINATA di stanze rappresentate da coppie (numero_piano,numero_settore)

f. Ufficio Prenotazioni deve poter prenotare una prestazione medica per pazienti esterni, che richiede una specializzazione “s”
    i. Il sistema deve restituire l’insieme dei medici maggiormente idonei a soddisfarla. Il criterio di idoneità è il seguente: se esistono medici con specializzazione primaria pari ad s, il risultato è l’insieme di tali medici. Altrimenti, il risultato è l’insieme dei medici che hanno “s” tra le loro specializzazioni secondarie
g. Tutti devono poter ottenere le specializzazioni primarie e secondari dei medici dell’ospedale

Diagrammi UML (miei)

- Indirizzo deve essere unico, ma cosa succede se due persone vivono nello stesso palazzo? Allora il tipo “indirizzo” deve essere molto preciso, deve includere il numero dell’interno. Ma non basta, perché se due pazienti vivono nella stessa stanza non li posso registrare
- Un medico può essere anche paziente?
- Un medico deve per forza avere delle spec. Secondarie?
- Le prestazioni mediche esterne devono essere sempre associato ad almeno un medico?
- Come dovrei separare i pazienti interni da quelli esterni?
- getItinerario() dovrebbe essere un’operazione della classe Medico?
- Itinerario deve essere una classe o un tipo di dato?
- Non so se conviene mettere un’operazione “getSpecializzazioni()” per la classe Medico

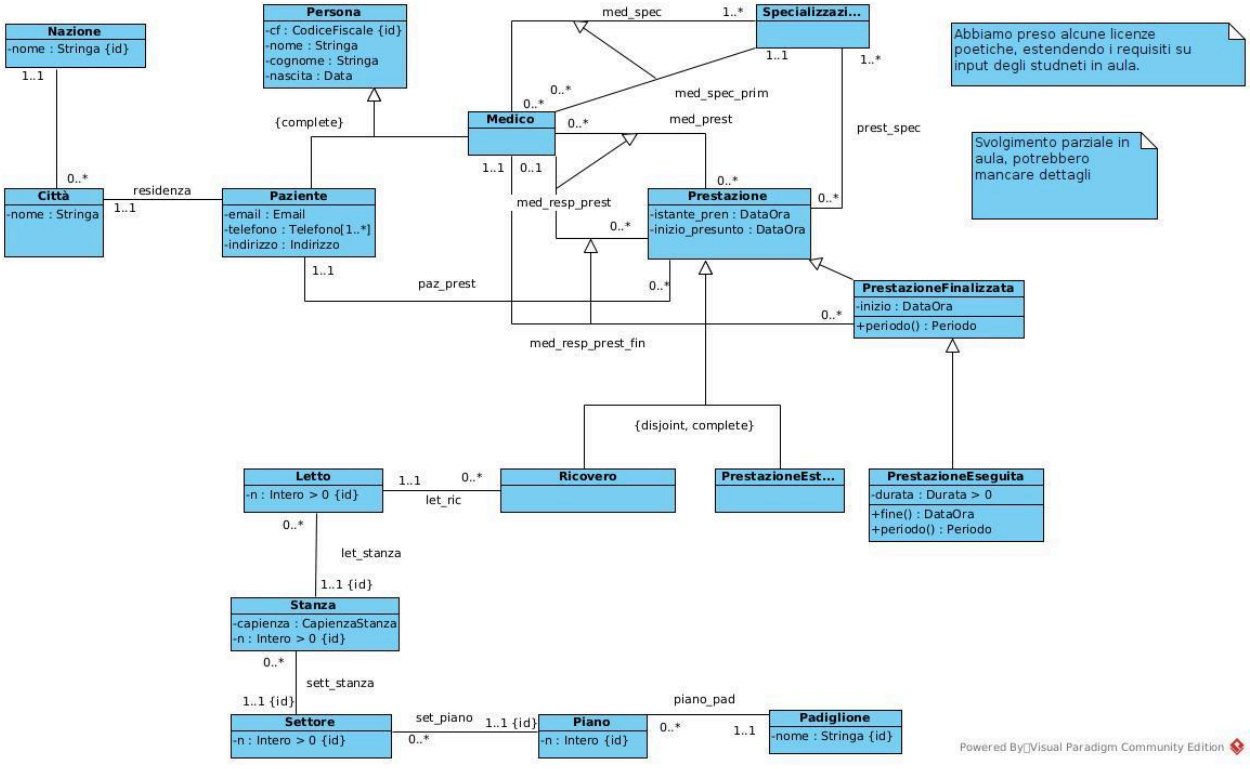


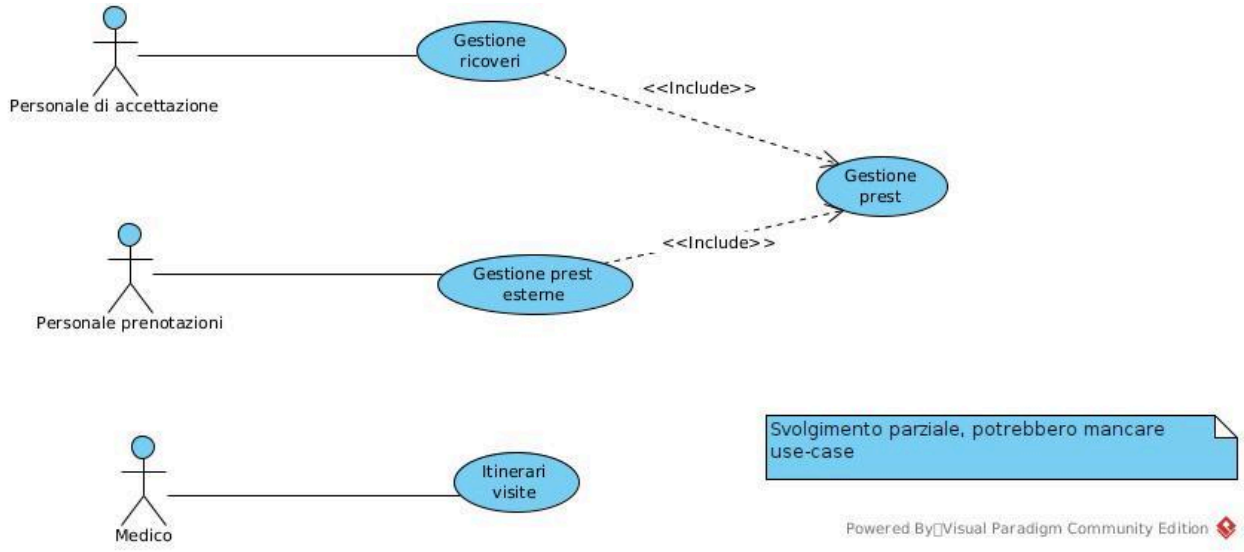


Diagrammi UML (Mancini)

- Non c’è distinzione specifica tra “Paziente” e “Paziente Esterno”
- Non esistono le operazioni
    - getItinerario
    - getSpecialPrimaria()
    - getSpecialSecondarie()
    - getRicoveriConclusi()

    - occupatoDa()
- Una prestazione può necessitare di più di una specializzazione, una prestazione può essere assegnata a più medici (di cui uno responsabile)
    - In particolare, per una prestazione non ancora confermata (ovvero non finalizzata) non è detto che ci sia un medico responsabile (0..1) o che almeno un medico sia coinvolto (0..*)
    - Dal momento che la prestazione è confermata (finalizzata) ci sarà sicuramente un medico coinvolto, e questo sarà il medico responsabile
- Anche i ricoveri possono essere prenotabili (ad es., per un intervento chirurgico pianificato)
- Le prestazioni (sia ricoveri che prestazioni esterne) seguono lo stesso flusso: prenotate (con istante di inizio presunto), finalizzate (con i medici assegnati ed un istante di inizio certo), eseguite (con un istante di fine)
- Non c'è il concetto di "Ricovero concluso"





Specifica dei tipi di dato (Mancini)

- Periodo
  - da: DataOra[0..1]
  - a: DataOra [0..1]
  - [V.Periodo.legale]
    - ALL p, x, y:
      Periodo(p) AND da(p,x) AND a(p,y)
      -->
      x<=y
  - interseca(p1: Periodo, p2:Periodo): booleano
    - #devo considerare che i periodi possono NON avere data di inizio e fine
    - Se entrambi i periodi non hanno né data di inizio né data di fine allora NON si intersecano
    - Se uno solo dei periodi ha data di inizio, allora NON si intersecano
    - Quindi result= false per default
    - Se entrambi i periodi hanno data di inizio ma non di fine allora si intersecano, ovvero

        - (EXISTS da1,da2: da(p1, da1) AND da(p2, da2))
          AND
          (NOT EXISTS a1,a2: a(p1,a1) AND a(p2,a2))
          →
          result = true
    - Se un periodo finisce dopo o proprio all'inizio dell'altro (che potrebbe non avere data di fine) allora si intersecano, ovvero
        - (EXISTS a1, da2: a(p1,a1) AND da(p2,da2) AND a1>=da2) -> result=true
          OR
          (EXISTS a2, da1: a(p2,a2) AND da(p1,da1) AND a2>=da1) -> result=true
    - Se entrambi i periodi hanno data di inizio e fine, allora si intersecano quando hanno un elemento in comune, ovvero
        - EXISTS x,da1,a1,da2,a2:
          da(p1,da1) AND a(p1,a1) AND da(p2,a2) AND a(p2, a2) AND (da1<=x<=a1) AND (da2<=x<=a2)
          →
          result=true
- CodiceFiscale: Stringa ~ '...'
- Email: Stringa ~ '...'
- Telefono: Stringa ~ '...'
- Indirizzo:
    - via: Stringa
    - civico: Stringa ~ '...'
    - cap: Stringa ~ '...'
- CapienzaStanza:
    - n_letti_tipico: Intero > 0
    - n_letti_max: Intero > 0
    - [V.CapienzaStanza.n_letti_max]
        - ALL c, t, m:
          CapienzaStanza(c) AND n_letti_tipico(c,t) AND n_letti_max(c,m)

→
t<=m

Specifica della classe Prestazione

- [V.Prestazione.istante_presunto_dopo_istante_pren]
  - …
- [V.Prestazione.medico_non_paziente]
  - Un paziente non può essere uno dei medici coinvolti in una sua prestazione richiesta, ovvero
    - NOT EXISTS m,prest:
      Medico(m) AND Paziente(m) AND paz_prest(m,prest)
      AND (med_prest(m,prest) OR med_resp_prest(m,prest))

Specifica della classe PrestazioneFinalizzata

- periodo(): Periodo
  - Pre: nessuno
  - Post:
    - result è un periodo che ha SOLO la data di inizio, e non quella di fine, ovvero
      result soddisfa:
      EXISTS result, i:
      $Periodo_{PrestazioneFinalizzata}$(this, result) AND da(result, i) AND inizio(this, i) AND (NOT EXISTS f: a(result,f))
- [V.PrestazioneFinalizzata.inizio_dopo_istante_pren]
  - …
- [V.PrestazioneFinalizzata.specializzazioni]
  - Il medico assegnato alla prestazione finalizzata (confermata) ha la specializzazione richiesta, ovvero
    - ALL prest, spec:

PrestazioneFinalizzata(prest) AND
Specializzazione(spec) AND prest_spec(prest, spec)
->
EXISTS m:
Medico(m) AND med_spec(m,spec) AND
med_resp_prest_fin(m,prest)

Specifica della classe PrestazioneEseguita

- fine(): DataOra
  - Pre: nessuna
  - Post:
    - result soddisfa la formula
      EXISTS i, d:
      inizio(this, i) AND durataMinuti(this,d) AND
      DataOra(result) AND (result=i+d)
- periodo(): Periodo
  - Pre: nessuna
  - Post:
    - result soddisfa la seguente formula
      EXISTS i,f:
      Periodo(result) AND inizio(this,i) AND da(result, i) AND
      $fine_{PrestazioneEseguita}$(this,f) AND a(result,f)

Specifica della classe Ricovero

- [V.Ricovero.no_due_ricoveri_simult_stesso_paziente]
  - NOT EXISTS p, r1, r2:
    Paziente(p) AND Ricovero(r1) AND Ricovero(r2) AND r1!=r2
    AND paz_prest(p, r1) AND paz_prest(p, r2) AND
    interseca(r1.periodo(), r2.periodo())

Specifica della classe Letto

- [V.Letto.no_due_ricoveri_simult]
    - NOT EXISTS l, r1, r2:
      Letto(l) AND r1!=r2 AND let_ric(l, r1) AND let_ric(l, r2) AND
      interseca(r1.periodo(), r2.periodo())

Specifica use-case “Gestione Prestazioni”

medici_idonei(s:Specializzazione): Medico [0..*]

- Pre: nessuna
- Post:
    - Sia M1 ={ m | Medico(m) AND med_spec_prim(m,s) }
    - Se M1 NON è vuoto, allora result=M1
    - Altrimenti, result ={m | Medico(m) AND med_spec(m,s)}

aggiungiMedico(p:Prestazione, m:Medico)

- Pre:
    - NOT PrestazioneEseguita(p)
- Post:
    - Nuove ennuple:
        - med_prest(m,p)

finalizzaPrestazione(p:Prestazione, i:DataOra)

- Pre:
    - (NOT PrestazioneFinalizzata(p))

        - ALL p_ist_pren: istante_pren(p, p_ist_pren) -> p_ist_pren < i
        - i>=adesso
        - EXISTS m: med_resp_prest(m,p)
        - Ogni specializzazione richiesta dalla prestazione, ha un medico che ce l'ha e che è coinvolto nella prestazione, ovvero
            - ALL s: prest_spec(p,s) → EXISTS m: med_prest(m,p) AND med_spec(m,s)
- Post:
    - Sia m tale che med_resp_prest(m,p)
    - Nuove ennuple:
        - PrestazioneFinalizzata(p)
        - inizio(p, i)
        - Il medico responsabile è ora confermato, ovvero med_resp_prest_fin(m, p)

registra_termine(p:Prestazione)

- Pre:
    - PrestazioneFinalizzata(p)
    - NOT PrestazioneEseguita(p)
    - EXISTS i: inizio(p, i) AND i<=adesso
- Post:
    - Sia i tale che inizio(p, i)
    - Nuove ennuple
        - PrestazioneEseguita(p)
        - durata(p, adesso - i)

Specifica use-case "Gestione Ricoveri"

medici_idonei(s: Specializzazione): Medico [0..*]

- PRE:
    - Nessuna
- POST:

- result = GestionePrestazioni.medici_idonei(s)

aggiungiMedico(r: Ricovero, m: Medico)

- PRE:
  - NOT PrestazioneEseguita(r)
- POST:
  - result = Gestione_prestazioni.aggiungiMedico(r, m)

Specifica use-case “Itinerari Visite”

itinerario(): (r: Ricovero, pos: Intero > 0) [0..*]

- PRE: nessuna
- POST:
  - Sia m:Medico l’oggetto associato all’attore che ha invocato l’operazione
  - Sia R = { r | Ricovero(r) AND (NOT PrestazioneEseguita(r)) AND med_prest(m,r)}
  - result= sorted(R, ricovero_LEQ)
  - Dove ricovero_LEQ(r1: Ricovero, r2: Ricovero)
    - Pre: nessuna
    - Post:
      - Siano p1, p2, sett1, sett2 tali da soddisfare:
        EXISTS l1,l2,sett1,sett2,st1,st2:
        let_ric(l1,r1) AND let_stanza(l1,st1) AND sett_stanza(sett1,st1) AND sett_piano(sett1,p1) AND let_ric(l2,r2) AND let_stanza(l2,st2) AND sett_stanza(sett2,st2) AND sett_piano(sett2,p2)
      - result =
        true
        <--->
        [p2>p1
        OR (p2=p1 AND sett2>=sett1)]

Ristrutturazione del diagramma delle classi

